Modeling Compositional and Non-compositional Compound-Word Semantics with a Feed-Forward Neural Network

Abstract
In this paper, we investigate whether a simple feed-forward neural network can both (1) compose the meanings of novel, compositional compound words by fusing constituent embeddings, and (2) memorize non-compositional ("idiomatic") compounds. Using 300-dimensional GloVe vectors for closed-form compounds, we train models that map the concatenation of two 300-d component word embeddings (600 d total) onto the target compound embedding of 300-d. We compare models with different hyperparameter designs, specifically we discuss the impact of linear vs. one-hidden-layer architectures, activation (linear vs. tanh), hidden-unit size (300, 150, 100), and learning rates (0.05, 0.01, 0.001). Performance is evaluated on held-out compounds for validation and analyzed by the performance errors within fixed epochs. Our best model achieves an RMSE of ≈ 0.23 per embedding coordinate for both the validation set and the testing set, and the performance for specific examples supports the hypothesis that a single network can learn both fusion rules and exceptions, and generalize the rule to unseen compositional compounds. We discuss implications for psychological theories of lexical retrieval and conceptual blending. Further study can compute compositionality of examples used through additive baseline models.

I. Introduction

Problem & Motivation. Human readers effortlessly retrieve meanings for both compositional compounds (e.g., "doghouse") by fusing constituent concepts and idiomatic, non-compositional compounds (e.g., "ladybird") by memorized lookup. How might a single, psychologically-plausible network simulate this retrieving process and accomplish both rule-governed composition and exception memorization semantically speaking? We simulate this with a feed-forward model trained on pre-trained GloVe embeddings.

Literature & Hypothesis. The Oxford Handbook[1] of Compounding distinguishes compositional vs. non-compositional word sequences in linguistic terms. Compositional compound words have a range of degrees of compositionality. All compositional compound words show at least some variability (e.g.: one of the constituents can be used in plural form, like red carpets) and literal senses (e.g.: each constituent retains its prototypical and context-free meaning), which allows their meanings to be derived from their sub-components. On the other hand, non-compositional compound word (e.g.: ladybird) are idiomatic expressions that are fixed in form and lack full literal transparency. Meanings are hard to be computed from their parts and must be memorized as a whole. Yet the theory provides no mechanistic account of how both types of compounds are handled in a unified system. We

[1] Bauer, L., Lieber, R., & Plag, I. (Eds.). (2010). *The Oxford Handbook of Compounding*. Oxford University Press.

hypothesize that a nonlinear simulation model with hidden layer can (a) learn the general rule of understanding the compositional compound words that requires fusing the meanings of its component words and (2) memorize the non-compositional compound words it has seen.

II. Simulations

2.1 Data & Preprocessing

Embeddings.

To represent each compound and their component words as a dense vector, we make use of the publicly available GloVe embeddings[2] (glove.6B.zip), which were trained on a large‑scale corpus combining the 2014 English Wikipedia dump (≈ 1.9 B tokens) with the Gigaword 5 newswire collection (≈ 4 B tokens). This corpus mix provides broad coverage of both general and more specialized vocabulary, ensuring that common closed‑form compounds (e.g., "doghouse," "greenhouse") are present in the vocabulary. The glove.6B.zip release includes four embedding dimensionalities—50, 100, 200, and 300. We select the 300‑dimensional ("300d") vectors for our simulations to maximize semantic and representational richness: higher dimensional embeddings suppose to capture subtler semantic distinctions, which may be critical for modeling the blending of two constituent concepts.

Compounds in different orthographic forms.

Even though the definition of compound words is straightforward: a word that is made up of two or more component words that, when combined together, have a single unit of meaning. There are three types of compound words based on their orthographic form:

1. Closed compounds – written as a single word with no space
    - Example: greenhouse, notebook, basketball
2. Open compounds – written as two separate words with a space
    - Example: ice cream, high school, post office
3. Hyphenated compounds – written with a hyphen between words
    - Example: mother-in-law, well-being, check-in

We choose closed compounds for this project because these are most likely to appear as a single whole unit in pre-trained embeddings sets like GloVe instead of being divided up into two separate words. Moreover, there is no need for additional parsing or phrase-level token handling.

Whitelist Filtering.

Not all closed strings in GloVe correspond to genuine compound words. Most of the words we filter out when directly looking for compound-like words that have their subparts also in the GloVe dataset are morpheme-based single words such as "un-done", because GloVe include a great number of morphemes' embeddings. so we began by compiling a curated "whitelist"[3] of high‑frequencyEnglish

[2] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. [pdf] [bib]
[3] "Compound Words List," ProofreadingServices.com, accessed May 5, 2025, https://www.proofreadingservices.com/pages/compound-words-list.

compounds (sourced from an online proofing service's list). A Python script then checks each candidate against the GloVe vocabulary lists: compounds or constituent words not found are discarded.

We collect and sample from the verified whitelist a total of 300 compound words and partition them into a training set (180 compound words), a held-out validation set (60 compound words) for tuning hidden layer size and other hyperparameters, and a separate test set (60 compound words) to evaluate generalization performance and detailed performance on each examples. Cross-validation will not be used due to the computational simplicity and interpretability focus of the simulation. Examples of the compound words used and their component words are documented in training.csv, validation.csv, and testing.csv. Their corresponding input and output embeddings are written in lens format in training_ex.txt, validation_ex.txt, and testing_ex.txt.

Input Construction.

For each retained compound word C composed of component words w_1 and w_2, we retrieve their 300 d GloVe vectors $\mathbf{v}_1$ and $\mathbf{v}_2$. We then concatenate them to form a single 600 d input vector $[\mathbf{v}_1; \mathbf{v}_2]$. This concatenation preserves both order and full representational detail about both component words without averaging them away, which would happen if we take the mean 300d vector as input. It also allows the network to learn asymmetric contributions (e.g., "note" vs. "book" in "notebook") rather than forcing a symmetric average (e.g., "note" contributes more than "book"). The target vector of each example is 300 d, and it is the compound word's actual embedding in GloVe.

2.2 Model Architectures

All models will learn a projection from the 600d input space to the 300d output space via mean-squared error (SSE) loss. Each run with the same infrastructure reuses a fixed random initialization to isolate hyperparameter effects. SSE is the natural choice of loss function for our model because we are doing a regression task more than a classification task, and we want the output vector to be as close as possible to the target embedding. SSE measures the squared distance between predicted and target vectors, which directly reflects this goal. SSE is also the default objective in LENS when specifying a continuous output task.

2.3 Experiment 1: Linear vs. Nonlinear Composition

Methods

- Compare linear vs. tanh (normalized) models.
- Learning rate = 0.05, epochs until convergence (< 0.05 improvement/epoch), no hidden layer.

Linear Model

Since some values of the word embeddings in all three datasets are above 1 and below -1, which can not be captured by the output range of many common activation functions, for example sigmoid with range (0,1), tanh with range (-1,1), it saturates at these bounds and gradients shrink on extremes, causing no valid update and error computation. The linear function (-inf, +inf) as activation matches the embedding value range without clipping. However, it has very poor performance. Using

compound-shallow-linear_in.txt, the training error sharply decreases to near 0 at around 80 epochs, but it actually keeps circling between a couple of error values and does not truly or smoothly converge. This could be a high-dimensional linear least squares problem, as the Hessian of our loss has many very small eigenvalues that act as flat directions. When the linear model starts circling around error values, the validation set performance is very poor with a total SSE error around 3000, and it only hits one example correctly (1.67%), meaning that the model has little generalization power for unseen examples. By looking into the specific compound words, we found out that compositional compounds in the validation set are captured poorly by the model.

Tanh Model with normalized inputs and targets

After observing that raw GloVe vector dimensions often exceed the (–1, 1) range—leading to saturation in bounded activations and vanishing gradients—we apply max-absolute scaling to every embedding vector before training the network. Concretely, for each 300 d vector v, we find out the maximum absolute value and divide each vector entry by that value. This guarantees that all components of the scaled vector v′ lie strictly within (–1, 1), so that subsequent tanh activations in the hidden layer can freely operate.

Running the compound-shallow-tanh_in.txt script, the model reaches the stopping threshold and is able to converge after ≈ 180 epochs. Training-set SSE declines smoothly until converging. Looking into the concrete examples, weights have adapted to both compositional patterns and memorized exceptions. Despite stable convergence on training data, the shallow-tanh network generalizes poorly under this setting: on the held-out 60-item validation set, it correctly "reaches" only 4 examples, which is an accuracy of 6.7 %, and has a total SSE error of around 2500. This gap between training fit and validation tests suggests that, without further architectural modifications, the model overfits to idiosyncratic embedding patterns of seen compounds rather than capture the true underlying pattern of compositionality and learning broadly applicable compositional rules. It is just memorizing the examples seen in the training set.

2.4 Experiment 2: Hidden Size & Bottleneck

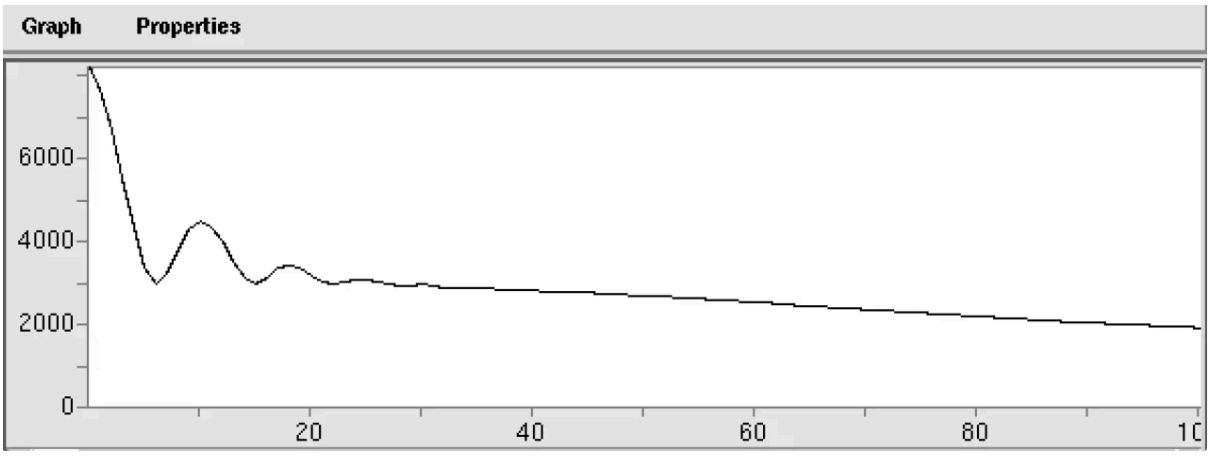
Methods

- Single Hidden = 300 vs 150 vs 100 (bottleneck).
- Learning rate = 0.05 vs 0.01 vs 0.001
- tanh activation, epochs until convergence (< 0.05 improvement/epoch)

Divergence of Training and Validation Performance

Starting with a configuration of 300 hidden units and a learning rate of 0.05, we trained a model with compound-hidden-300-0.05_in.txt, which starts at a total SSE of more than 8000. Shown by the error graph plotted below, the SSE experiences a damp oscillation and then turns into a very slow decay. Around 2500 epochs, the training SSE is able to converge. This pattern of error graph could be due to the fact that the learning rate is too high, or there are too many units in the hidden layer. With a high learning rate and a mismatched curvature of the loss surface, a large step size can cause the loss optimizer to overshoot the minimum along steep axes. The successive weight updates then bounce back and forth across the valley floor, which produces the characteristic damped oscillation pattern.

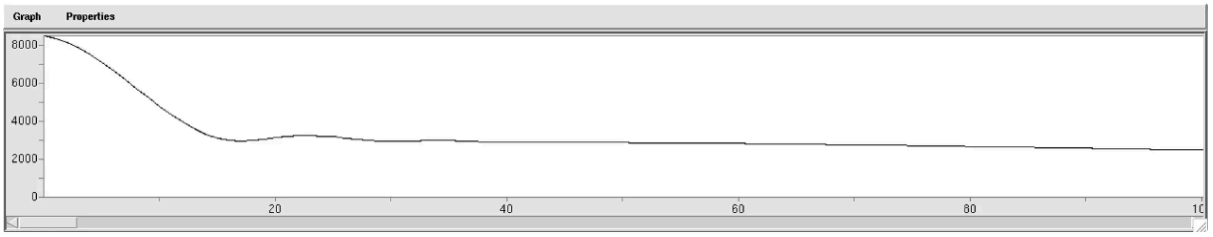
Large layer of hidden units often has a wide spectrum of eigenvalues, which worsens oscillation. [4]



The poor performance on the validation set not only demonstrates the problems in the trained model but also reveals another phenomenon: **At 100 epochs**, the total training error is around 2235, whereas the total validation error is around 961, reaching 32 held-out validation compounds (53.33%). **At 200 epochs**, the total training SSE has fallen to about 1512.29, whereas the validation SSE rises to 1021.47, with the model "reaching" 34 of the 60 held-out compounds (56.67 %). **At 300 epochs**, the training SSE further decreases to approximately 1066.78, but the validation SSE rises slightly to 1131.00, and only 25 compounds (41.67 %) meet the hit criterion. **At 500 epochs**, training SSE reaches about 601.99, yet validation SSE climbs to 1334.89, with just 20 compounds (33.33 %) correctly predicted. When training performance has converged **at around 2500 epochs**, the training SSE keeps at around 32.04, whereas the total SSE error of the held-out validation set is around 2431.64 with an error per example at around 40. We reached only 1 example with a total accuracy of (1.67%). The training and testing error is diverging at early epochs. There is a rapid fit to the idiosyncrasy in the training set, such that the network is purely memorizing at an early point rather than capturing the general rule for compositional words. Excessive capacity like 300 hidden units can memorize nearly every training example's unique embedding pattern as there is too much memory space for storing the specific instance. These infrastructure do not hold for unseen compounds. A moderately high learning rate amplifies divergence once it starts chasing small idiosyncratic gradients.

Failed Attemps

With configurations like 300 hidden units and 0.01 learning rate documented in compound-hidden-300-0.01_in.txt, the convergence rate of the training set gets enhanced at around 1300 epochs, and convergence is reached more smoothly without damp oscillation.



Nevertheless, the performance on the held-out validation set is still problematic. Specifically, the divergence of the training set and validation set performance continues to exist.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning* (Cambridge, MA: MIT Press, 2016), sec. 8.3.

Table 1. Performance for compound-hidden-300-0.01_in.txt (300 hidden units, η=0.01)

Updates (epochs)	Training SSE	Validation SSE	Validation "Reached" Examples
100	4611.34	2150.00	35 / 60 (58.3 %)
200	2813.82	1868.00	32 / 60 (53.3 %)
500	469.23	2891.00	11 / 60 (18.3 %)

Other configurations, such as a model with 150 hidden units and 0.01 learning rate, or one with 100 hidden units and 0.01 learning rate, all remain the divergence performance pattern.

Table 2. Performance for compound-hidden-150-0.01_in.txt (150 hidden units, η=0.01)

Updates	Training SSE	Validation SSE	Reached Examples (%)
100	2 530.72	946.17	38 / 60 (63.33 %)
200	1 747.46	986.12	36 / 60 (60.00 %)
300	1 156.96	1 121.08	25 / 60 (41.67 %)

Table 3. Performance for compound-hidden-100-0.01_in.txt (100 hidden units, η=0.01)

Updates	Training SSE	Validation SSE	Reached Examples (%)
100	2 533.04	939.45	38 / 60 (63.33 %)

200	1 816.34	982.23	36 / 60 (60.00 %)
300	1271.82	1095.36	27/60 (45.00%)

Best Model So Far

The best model we have after multiple hyper-tuning is the configuration of 100 hidden units and 0.001 learning rate.

Table 4. Performance for compound-hidden-100-0.001_in.txt (100 hidden units, η=0.001)

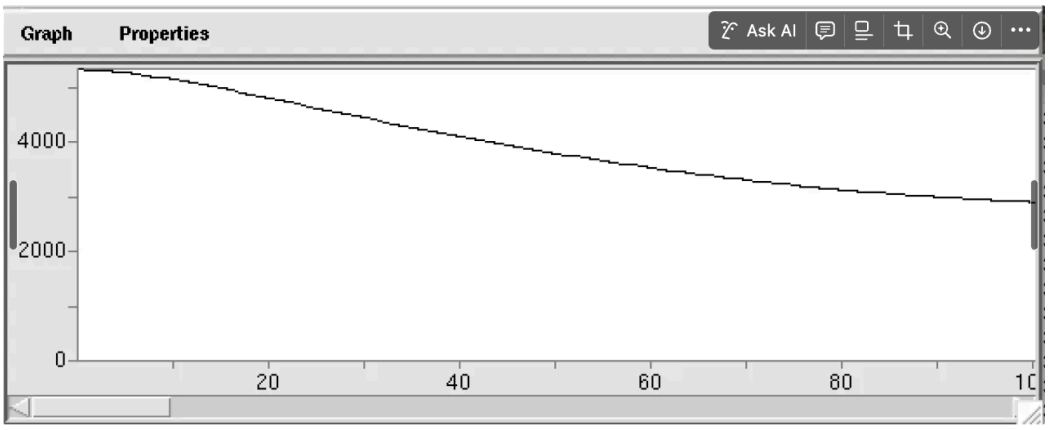
Updates	Training SSE	Validation SSE	Reached Examples (%)
100	2 896.03	962.34	35 / 60 (58.33 %)
200	2 813.84	952.64	38 / 60 (63.33 %)
400	2 663.71	944.09	40 / 60 (66.67 %)
700	2 428.42	936.88	42 / 60 (70.00 %)

The 100-unit, η=0.001 model clearly outperforms all prior configurations: unlike the earlier runs where validation SSE quickly gets to the bottom at around epoch 100 then climbed steadily, this small, and low learning rate network exhibits monotonic improvement on held-out compounds throughout the training process from 0 epochs to around 1000 epochs. At 100 updates its validation SSE (962.34) and reach rate (58.3 %) already match the best validation performance of the 300 hidden units, η=0.01 model, but by 700 epochs of updates, its SSE has dropped to 936.88 while its hit-rate climbs to 70 % (about the greatest accuracy of all). The training and validation curves remain in the same decreasing trend for another hundreds of epochs, even though the validation performance almost converges, indicating that the model has found a stable compositional rule, rather than merely memorizing training examples, which helps to generalize to unseen compounds. The lowest total SSE for the held-out validation is around 936.88, with the SSE per example being 15.61. The Mean Squared Error (MSE) per dimension is therefore SSE per example divided by each example's dimension of 300, which is about 0.052. The Root Mean Squared Error (RMSE) per dimension is about 0.228, while RMSE ~0.35 is often reported as a representative value for an unregularized additive baseline, which

assumes that meanings of two component words “adds up” roughly linearly to form a constitutional compound.[5]

2.5 Test Results

We revisit the trained model of 100-unit, η=0.001 and run it on the testing set. The training error curve descends smoothly, but it has not reached its asymptote at the prescribed 700 epochs. At 700 epochs, the network hits 31 examples with an accuracy of 51.67% for the testing set. The total testing SSE is around 945.29, and the SSE per example is around 15.75. The MSE per dimension is therefore about 0.053, and the RMSE per dimension is about 0.229. This RMSE value represents a substantial improvement over our earlier shallow-tanh and higher-η models, which typically settled around 0.35–0.40.

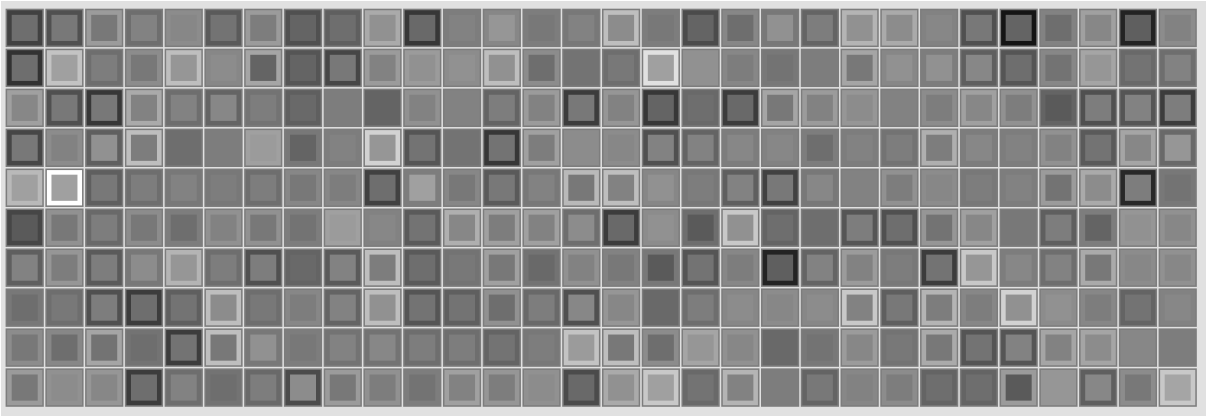


Looking at each specific examples in the Unit Viewer and examine the hypothesis we made, we figure out a clear pattern of both the compositional and non-compositional compound examples in the training set have outputs very close to their target values, which shows that PDP-style models are good at regularity and exception coexistence.

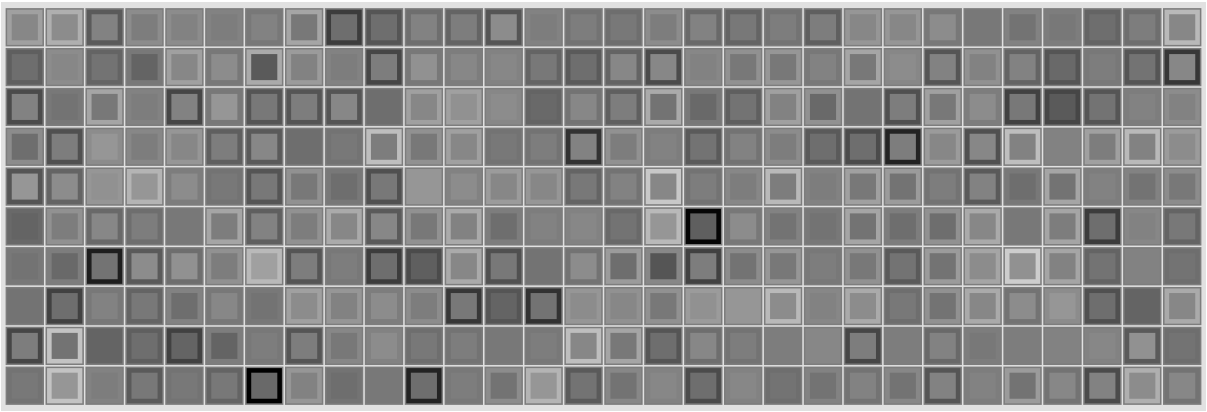
Below is the visualized output pattern of the word “landowner” in the training set in Black-Grey-White Diagram. It is a compound word that we can easily derive its entire meaning from its components “land” and “owner”. We can see that most output units are close to their targets with some noise and idiosyncrasies that have not yet been fully memorized. Other than this word, many compositional compounds have most units’ output settle down at early stages. This suggests the

[5] Mitchell, J., & Lapata, M. (2008). Vector-based models of semantic composition. *Proceedings of the 46th Annual Meeting of the Association for Computational Linguistics*, 236–244.

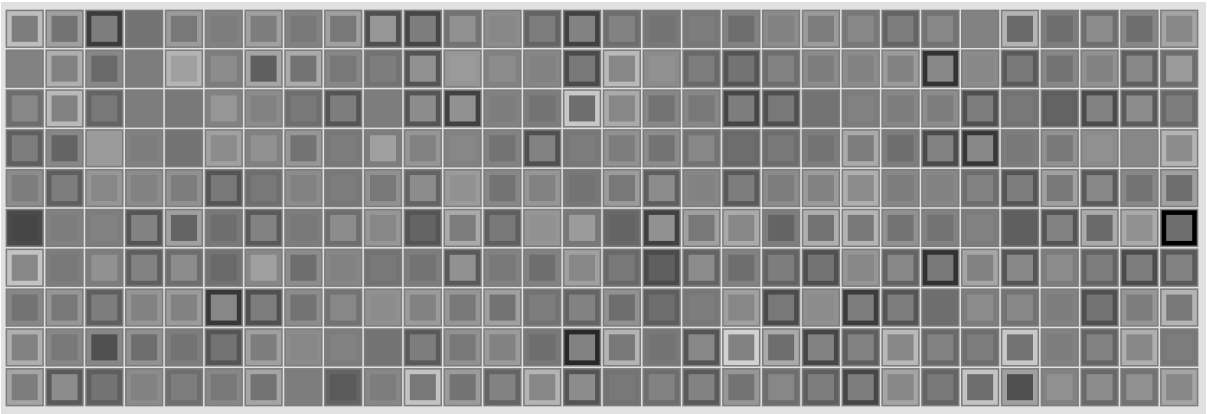
model's potential to use the general compositional rule to retrieve the compound's meaning.



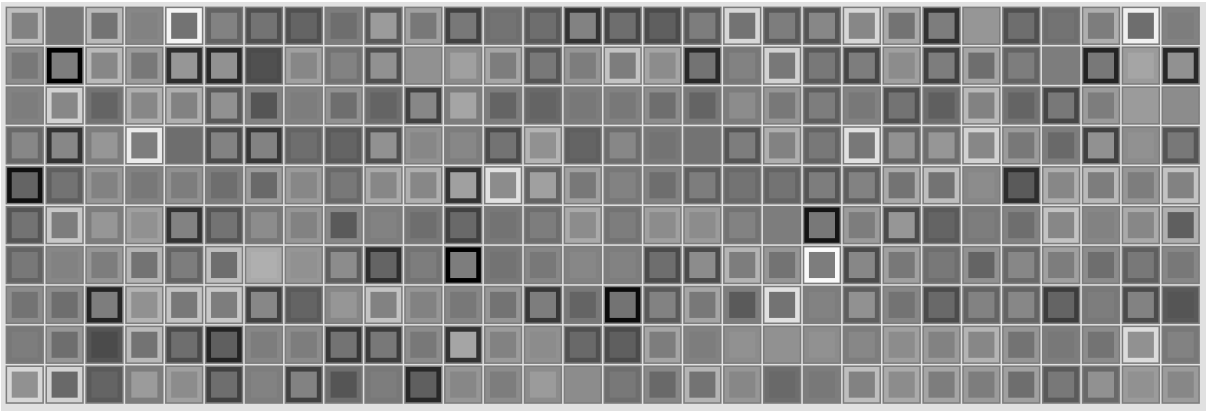
The below output pattern showing the predicted embedding for "livestock" is an example of the model handling exceptions. "livestock" is not very compositional as its meaning is a little hard to infer just by looking at the semantic meanings of "live" and "stock". Still, most units of the output pattern hit a similar value to their targets, showing the model's capacity of learning exceptional expressions that requires some semantic memorization of the entire word.



Now comparing the above to the individual performance on the testing sets. We first look at the word "watertight", which is a somehow compositional compound that alludes to the image of "water" and "being tight". The output performance shows that the model captures most of the units' embedding values, and these well-modeled units are likely to be the consistent semantic meaning of "watertight" from "water" and "tight".



Below is the output performance of the word “limelight” in the testing set. This semi-to-none compositional word is a completely unseen example for the trained model. The general rule of computing the total component embeddings and transforming it into the compound embedding should have lost its power when it comes to this word. Compared to “watertight”, this example with poorer output performance shows that the model is not randomly guessing the value of each output unit.



III. General Discussion

We have shown that even a simple one-hidden-layer network can both fuse constituent embeddings for coherent representations of unseen compositional compounds and memorize non-compositional exceptions when trained on a small set. Nonlinearity is crucial for learning the general constitutional rule. The nonlinear units improve compositional generalization, indicating the network learns conceptual blending beyond simple projection. Hidden-unit capacity modulates the balance between rule-based generalization and exception memorization. Large hidden-unit layers can make the model easily overfit to idiosyncratic training noise, which causes validation error to rise sharply with continued training. High learning rate can lead to damped oscillations and slow decay, causing the network to overshoot the steep directions and also worsen overfitting.

Psychological Implications. Our results support hybrid cognitive models in which the brain flexibly engages both distributed rule-based learning and item-specific memory. The semantic richness (300d) of the two components means that two concepts are fully activated in working memory waiting for semantic fusion, while bottleneck constraints may correspond to working-memory limits: larger “hidden capacity” supports richer compositional operations.

Future Directions.

-Expand the dataset sizes and avoid the dominance of non-compositional words**,** under which the model could hardly learn the general rule of compositionality.

- Extend to phrase-level tokens (open compounds) with contextualized embeddings.

- Quantify the “compositionality” of each compound by using additive or multiplicative baseline model to output a 300d purely compositional embedding for a pair of components and computing the

cosine similarity of this embedding to the target 300d embedding. Study the correlation between compositionality and the performance measure.